



中国矿业大学  
CHINA UNIVERSITY OF MINING AND TECHNOLOGY

# 《大数据可视化》综合设计报告二

## 题目：移动端可视化实践

学 院 计算机科学与技术学院

班 级 2023 级大数据一班

姓 名 王志豪

学 号 08232337

中国矿业大学

2026 年 03 月

课程设计报告格式要求：

1. 报告中应包含一、二级标题；
2. 正文宋体小 4 号，1.5 倍行距；
3. 报告正文不少于 1000 字；
4. 上交报告电子版：<https://v.wjx.cn/vm/mv3f28U.aspx>，截止 4 月 26 日。

课程设计报告评分要点：

1. 报告中可视化技术的应用情况（20%）
2. 报告中可视化应用功能有效及完整性（40%）
3. 报告中的个人总结及体会情况（20%）
4. 报告格式规范程度（20%）

评语：

评分：

日期：

# 目录

|                    |    |
|--------------------|----|
| 1. 背景及功能概要.....    | 3  |
| 1.1 背景.....        | 3  |
| 1.2 功能概要.....      | 3  |
| 1.3 项目结构与使用指南..... | 4  |
| 2.可视化系统设计.....     | 6  |
| 2.1 概要设计.....      | 6  |
| 2.2 功能设计.....      | 7  |
| 3.可视化实现及验证.....    | 12 |
| 3.1 代码分析.....      | 12 |
| 3.2 运行截图.....      | 16 |
| 4.总结及体会.....       | 21 |

# 1. 背景及功能概要

## 1.1 背景

随着现代生活节奏的加快，人们的饮食结构日益复杂，健康管理需求不断增长。然而，传统饮食记录方式存在显著痛点：手动输入每餐食物名称、分量和热量操作繁琐，普通用户难以准确估算食物营养成分，缺乏有效的饮食趋势数据反馈，导致多数用户难以坚持记录。市面主流饮食管理应用以手动记录为主，AI 辅助能力弱，操作路径长，用户体验不佳。

NutriAI 正是在这一背景下诞生的 AI 辅助饮食管理应用。项目的核心目标是利用多模态大模型技术，以"拍照识食"为核心交互入口，将饮食记录的时间成本从分钟级压缩到秒级。用户只需拍摄餐食照片即可自动完成食物识别、营养估算、数据记录的全流程。项目采用前后端分离架构，前端基于 Android 平台使用 Kotlin 和 Jetpack Compose 构建 Material 3 风格的现代界面，后端基于 Python FastAPI 框架提供完整的 REST API 服务，数据库采用 PostgreSQL 并通过 Docker 部署。AI 能力通过接入 Gemini 多模态大模型实现，能够对食物图片进行结构化分析并返回完整的营养数据。

## 1.2 功能概要

NutriAI 当前版本实现了完整的饮食记录与营养分析闭环，核心功能涵盖以下模块：

- (1). **AI 图片分析**是项目的核心功能。用户通过拍照或从相册选取食物图片，上传至后端后自动调用 Gemini 大模型进行识别分析。AI 能够识别图片中的食材种类和份量，估算热量、蛋白质、碳水化合物和脂肪等营养成分，同时生成健康评分（A/B/C 三级）并提供每项食物的健康替代推荐。分析结果以结构化 JSON 返回，自动创建餐食记录并存入数据库，前端直接渲染展示。
- (2). **餐食管理功能**支持完整的增删改查操作。用户可以手动创建餐食记录，编辑餐食标题、类型、日期、时间、备注等信息，管理每餐的明细项目（包含食物名称、各项营养值和份量），删除不需要的餐食或明细项，以及将已有餐食复制到其他日期快速复用。每项明细都支持 AI 生成或手动添加的健康替

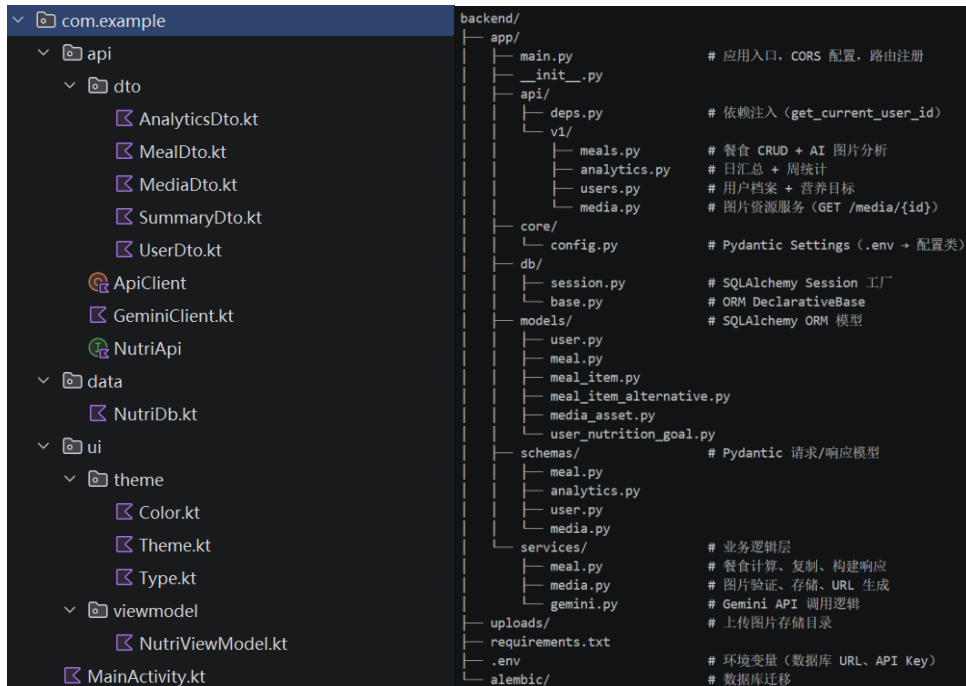
代推荐。

- (3). **周统计与分析功能**以周为维度展示营养趋势。后端通过 SQL 聚合查询计算七日数据，前端以柱状图形式展示热量、蛋白质、碳水化合物和脂肪的每日摄入量，并与基于用户身体档案计算得出的营养目标进行对比，超出目标的柱子自动标红。同时提供周汇总信息，包括日均值、记录天数和宏量营养素供能比例。
- (4). **日记功能**以照片网格形式（每行三列）展示所有带图片的餐食记录，支持按日期切换查看。日期系统基于北京时间动态计算，首页日期选择器显示今日及过去十天，所有日期均无硬编码，确保在不同时区和日期下正确显示。

## 1.3 项目结构与使用指南

项目分为前端和后端两个独立部分。前端目录存放于独立的 `frontend` 文件夹中，为 Android 原生应用；后端位于项目根目录的 `backend` 文件夹中，为 Python FastAPI 服务。

前端基于 Android 平台，使用 Kotlin 语言和 Jetpack Compose 声明式 UI 框架，采用 Material 3 设计规范。项目为单 Activity 架构，所有 UI 组件集中在 `MainActivity.kt` 中，通过 `NutriViewModel` 统一管理全局状态。网络层使用 `Retrofit + OkHttp + Moshi`，本地缓存使用 `Room Database`，图片加载使用 `Coil`。前端所需的最低 API 级别为 34（Android 14），建议使用 Android Studio 打开并运行。



后端技术栈为 Python 3.12+、FastAPI、SQLAlchemy 2.0 和 PostgreSQL 16。项目采用分层架构，分为路由层（api/v1/）、服务层（services/）、数据访问层（models/）和 Schema 层（schemas/）。

### 使用指南：

部署方式为 Docker 运行 PostgreSQL，通过 Uvicorn 启动 FastAPI 服务。

后端启动步骤为：

- (1). 首先执行 `docker run -d --name nutriai-postgres -e POSTGRES_PASSWORD=password -p 5432:5432 postgres:16` 启动数据库；
- (2). 然后执行 `docker exec -it nutriai-postgres createdb -U postgres nutriai` 创建数据库；
- (3). 最后在 backend 目录下执行 `pip install -r requirements.txt && uvicorn app.main:app --reload --host 0.0.0.0 --port 8000` 启动服务。后端启动后会自动播种测试用户数据，通过 `http://localhost:8000/docs` 可访问 Swagger API 文档。

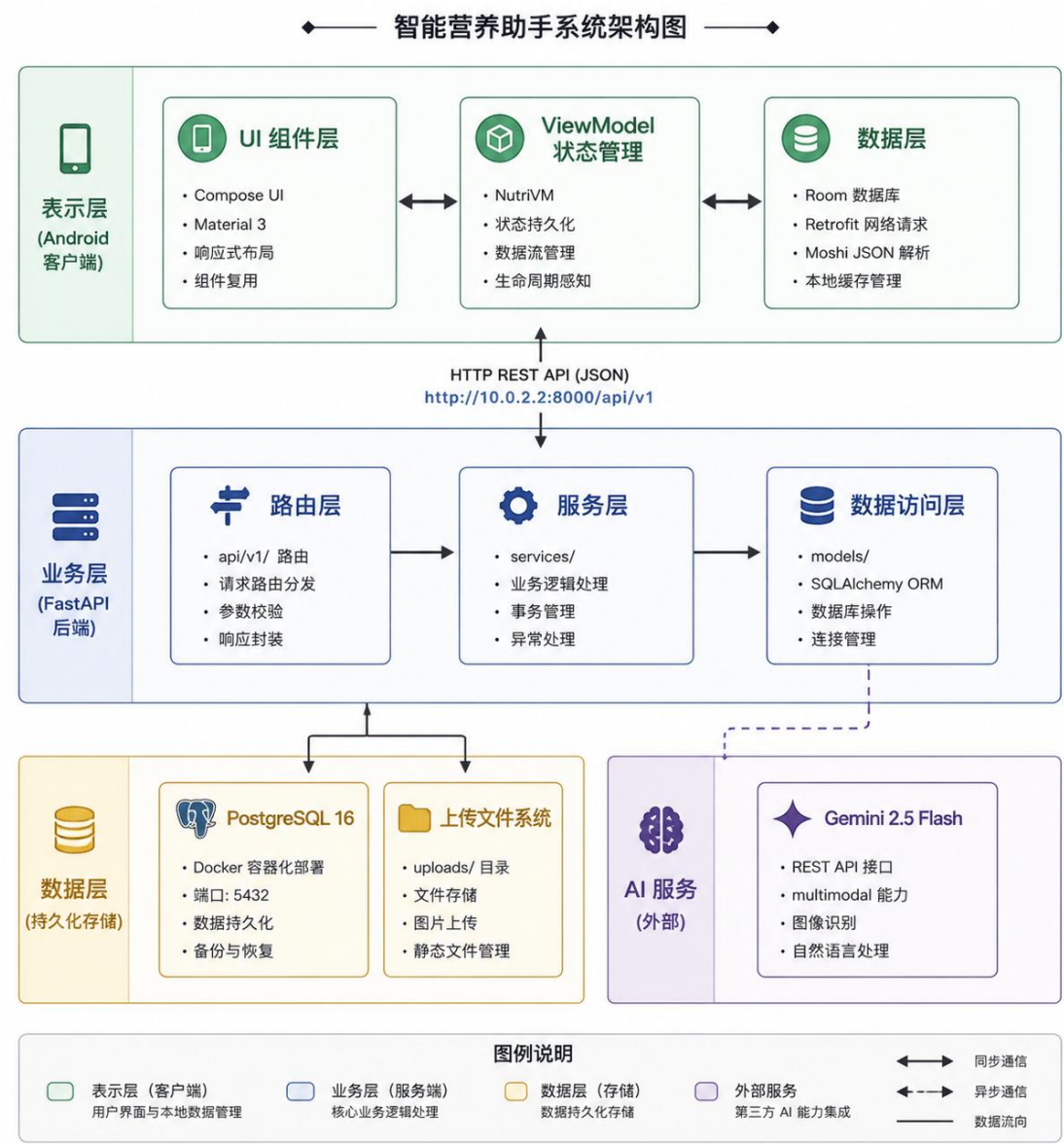
前后端通信通过 HTTP REST API 进行，Android 模拟器使用 10.0.2.2:8000 作为后端地址（该地址映射宿主机 localhost），真机调试需替换为宿主机局域网

IP。AI 分析功能需在 .env 文件中配置 GEMINI\_API\_KEY。

## 2.可视化系统设计

### 2.1 概要设计

NutriAI 采用经典的**前后端分离架构**，整个系统划分为**表示层、业务层和数**  
**据层**三个层次，层与层之间通过标准化的接口进行通信，职责边界清晰。



(1). **表示层**即 Android 客户端，运行在用户设备上，负责所有用户界面渲染和交互响应。该层采用 MVVM 架构模式，View 层由 Jetpack Compose 声明式 UI 组件构成，通过观察 ViewModel 层暴露的 StateFlow 实现响应式更新。

ViewModel 层 (NutriViewModel) 集中管理全部 UI 状态, 包括当前选中的日期、餐食列表、分析状态、用户档案等, 并通过调用 Repository 层接口获取数据。数据来源分为本地 Room 数据库 (离线缓存) 和远端 Retrofit 服务 (后端 API)。表示层不承担任何 AI 计算任务, 仅负责数据解析和渲染展示。

- (2). **业务层**即 FastAPI 后端服务, 部署在服务器上, 处理所有业务逻辑和 AI 调用。该层内部进一步细分为路由层、服务层和数据访问层。路由层 (api/v1/) 负责 HTTP 请求接收、参数校验、权限检查和响应返回, 使用 FastAPI 的依赖注入系统自动解析请求参数和数据库会话。服务层 (services/) 封装具体业务逻辑, 包括餐食 CRUD 业务 (meal.py)、图片校验与存储 (media.py)、Gemini API 调用 (gemini.py)、营养目标计算 (nutrition.py) 和周统计聚合 (analytics.py)。数据访问层 (models/) 通过 SQLAlchemy ORM 定义数据库表结构与关系, 屏蔽底层 SQL 细节。三层各司其职, 路由层不包含业务逻辑, 服务层不直接处理 HTTP, 数据访问层不感知上层业务。
- (3). **数据层**由 PostgreSQL 数据库、本地文件系统和外部 Gemini API 三部分组成。PostgreSQL 通过 Docker 部署, 存储用户档案、餐食记录、餐食明细、替代推荐、图片资源元信息和营养目标等结构化数据。文件系统存储上传的原始图片文件, 通过 MediaAsset 表建立索引。Gemini API 作为外部 AI 服务, 接收 base64 编码的图片数据, 返回结构化 JSON 格式的营养分析结果。数据库所有表的主键采用 UUID v4 字符串策略, 便于前端预生成 ID 和后续分布式扩展。
- (4). **通信协议**方面, 前后端通过 HTTP REST API 进行数据交换, 请求和响应均使用 JSON 格式。后端 Pydantic Schema 层自动将 Python snake\_case 字段序列化为 JSON camelCase, 与前端 Kotlin 数据类直接对应。客户端通过 OkHttp 拦截器统一管理请求日志和错误处理, 通过 Retrofit 接口注解定义端点路由和序列化规则。

## 2.2 功能设计





```
{
  "title": "卤味鸡腿饭",
  "mealType": "午餐",
  "calories": 719,
  "protein": 35,
  "carbs": 68,
  "fat": 32,
  "healthScore": "B",
  "healthMessage": "蛋白质充足，碳水适中，建议增加蔬菜",
  "items": [
    {
      "name": "卤鸡腿",
      "calories": 250,
      "protein": 28,
      "carbs": 2,
      "fat": 15,
      "weightString": "1 只, 150克",
      "alternatives": ["去皮鸡腿 (更低脂)", "烤鸡腿 (少油)"]
    },
    {
      "name": "米饭",
      "calories": 240,
      "protein": 4,
      "carbs": 52,
      "fat": 0.5,
      "weightString": "1 碗, 200克",
      "alternatives": ["糙米饭 (高纤维)", "杂粮饭 (低升糖)"]
    }
  ]
}
```

- (1). **AI 图片分析功能**是系统的核心业务入口，其设计贯穿前后端四个层次。前端提供拍照（CameraX TakePicture）和相册（ActivityResultContracts.GetContent）两种图片获取方式，用户选择图片后，

NutriViewModel 调用 `analyzeImageMeal()` 方法，将图片压缩至 3MB 以内后通过 Retrofit Multipart 请求发送至后端 `POST /api/v1/meals/analyze-image` 端点。后端接收到图片后执行多步处理：首先校验图片格式（MIME 类型白名单 `image/jpeg`、`image/png`、`image/webp`）和文件大小（上限 10MB），然后生成 UUID 文件名按用户分目录存储至 `uploads/` 文件系统，同时在数据库中创建 `MediaAsset` 元信息记录。接着通过 `httpx` 异步 HTTP 客户端将图片 Base64 编码后发送至 Gemini REST API，请求中携带精确的 System Prompt 约束输出格式，关键配置包括 `responseMimeType: application/json`（强制输出合法 JSON）和 `temperature: 0.2`（保证输出稳定性）。Gemini 返回的结构化 JSON 包含餐食标题、类型、总热量、蛋白质、碳水、脂肪、健康评分和评语，以及每项食物的明细营养数据和健康替代推荐。后端解析后将数据拆分写入三张 ORM 表（`meals` 主表、`meal_items` 明细表、`meal_item_alternatives` 替代推荐表），调用 `recalculate_meal_totals()` 从明细求和回写主表合计字段，最后返回完整的 `MealDetail` JSON。如果 Gemini 调用失败，后端执行事务回滚撤销已创建的 `MediaAsset` 记录，确保数据一致性。前端收到响应后弹出 `MealReviewSheet` 底部详情页，展示所有营养数据和替代推荐，用户确认后记录正式生效。



- (2). 餐食管理功能遵循标准的 RESTful 设计，覆盖餐食和明细两个层级的 CRUD 操作。创建餐食时，前端提交 MealCreate JSON，后端在校验枚举值（meal\_type 限定 breakfast/lunch/dinner/snack，source 限定 ai\_image/manual/search\_history 等）和日期格式后，使用 SQLAlchemy ORM 写入数据库。餐食主表与明细表通过外键关联，每次写入或更新明细后自动调用 recalculate\_meal\_totals() 重新计算合计值并持久化到主表，采用预计算策略而非查询时聚合，以优化统计查询性能。删除餐食时级联删除关联明细和替代推荐，但保留最后一项时拒绝删除并要求改为删除整餐，防止数据残缺。餐食复制功能通过 duplicate\_meal() 服务实现深度克隆，遍历源餐食的所有明细和替代推荐创建新记录，支持指定新日期和来源标记。历史搜索通过 SQL LIKE 模糊匹配餐食标题和明细名称，按更新时间倒序返回。



- (3). 周统计与分析功能基于用户的详细身体档案进行个性化营养目标计算。后端通过 Mifflin-St Jeor 公式计算基础代谢率 ( $BMR = 10 \times \text{体重} + 6.25 \times \text{身高} - 5 \times \text{年龄} + \text{性别系数}$ )，乘以活动系数（久坐 1.2 至高强度 1.9）得到每日总消耗 TDEE，再根据健康目标（减脂 0.85、维持 1.0、增肌 1.1）调整得出目标热量。宏量营养素分配算法为：蛋白质按体重比例计算（减脂 1.8g/kg、维持 1.6g/kg、增肌 2.0g/kg），脂肪占总热量 25%，剩余热量分配给碳水化合物。周统计接口 `GET /api/v1/analytics/week` 接收 `start`、`end` 和 `metric` 参数，执行 SQL 聚合查询按日期分组求和，利用复合索引 (`user_id`, `meal_date`) 加速过滤。关键设计在于日期补全逻辑——SQL 查询仅返回有记录的天，后端通过循环遍历 7 天将无记录的天填充为 0，确保前端始终收到完整的 7 个数据点用于柱状图渲染。

```
uploads/  
└─ {user_id}/  
   └─ a1b2c3d4-e5f6-7890-abcd-ef1234567890.jpg  
   └─ b2c3d4e5-f6a7-8901-bcde-f12345678901.png  
   └─ ...
```

- (4). 图片存储功能在 MVP 阶段采用本地文件系统方案。上传时通过 `generate_storage_key()` 生成 `{user_id}/{uuid}.{ext}` 路径格式，按用户分目录存储避免文件数量过多。MediaAsset 表记录文件的 `content_type`、`size_bytes`、`storage_key` 和 `image_url` 等元信息，其中 `image_id` 取文件名 `stem` 便于关联。访问时通过 `GET /api/v1/media/images/{id}` 端点查询

MediaAsset 记录，拼接完整文件路径后通过 FastAPI FileResponse 返回。  
MEDIA\_BASE\_URL 配置为 http://10.0.2.2:8000，这是 Android 模拟器映射宿主机 localhost 的特殊地址，真机调试需替换为局域网 IP。

### 3.可视化实现及验证

#### 3.1 代码分析

以下从前后端各层挑选核心代码进行技术分析，从架构和工程角度论述其设计思路。

##### (1). 前端 ViewModel 层：状态管理与 AI 分析流程

```
fun analyzeImageMeal(imageUri: Uri) {
    _isAnalyzing.value = true // 触发 AnalyzingOverlay 显示
    viewModelScope.launch {
        try {
            val bytes = compressImage(context, imageUri, maxSizeBytes =
3_000_000)
            val mimeType = context.contentResolver.getType(imageUri) ?:
"image/jpeg"
            val requestBody = bytes.toRequestBody(mimeType.toMediaType())
            val imagePart = MultipartBody.Part.createFormData(
                "file", "nutriai_${UUID.randomUUID()}.jpg", requestBody
            )
            val sourcePart =
"camera".toRequestBody("text/plain".toMediaType())
            val response = api.analyzeImage(imagePart, sourcePart)
            if (!response.isSuccessful) {
                Log.e(TAG, "API error: ${response.code} ${response.message()}")
                return@launch
            }
            val detail = response.body() ?: return@launch
            // 映射后端 MealDetail 为前端 Meal 模型
            val foodItems = detail.items.map { item ->
                FoodItem(
                    name = item.name,
                    calories = item.calories,
                    weightString = item.weightString,
                    alternatives = item.alternatives.map { it.name }
                )
            }
            val parsedMeal = Meal(
```

```

        id = detail.id, title = detail.title,
        mealType = detail.mealType, mealTypeLabel =
detail.mealTypeLabel,
        mealDate = detail.mealDate, dateString = detail.dateString,
        timeString = detail.timeString,
        calories = detail.calories, protein = detail.protein,
        carbs = detail.carbs, fat = detail.fat,
        imageUrl = detail.imageUrl,
        healthScore = detail.healthScore,
        healthMessage = detail.healthMessage,
        itemsJson =
Moshi.Builder().build().adapter(List::class.java).toJson(foodItems)
    )
    openMealForReview(parsedMeal) // 弹出确认弹窗
} catch (e: Exception) {
    Log.e(TAG, "分析失败", e)
} finally {
    _isAnalyzing.value = false // 隐藏 AnalyzingOverlay
}
}
}

```

这段代码体现了三层设计考量。第一是用户交互层面，通过 `_isAnalyzing` 状态驱动 `AnalyzingOverlay` 全屏遮罩的显示与隐藏，该遮罩由三层动效叠加构成（旋转搜索图标、脉冲光环、`CircularProgressIndicator`），让用户感知系统正在运行而非卡死。第二是数据流层面，前端仅做图片上传后端的响应解析，将后端返回的 `MealDetail` 映射为前端 `Meal` 模型后直接渲染，不执行任何 AI 计算，这保证了 API Key 的安全性和业务逻辑的一致性。第三是容错层面，对 HTTP 错误码和异常进行捕获并日志记录，分析完成后无论成功或失败均在 `finally` 块中关闭加载状态。

## (2). 前端 UI 组件：NutriRing 热量环

```

@Composable
fun NutriRing(consumed: Int, target: Int) {
    val progress by animateFloatAsState(
        targetValue = (consumed.toFloat() / target).coerceIn(0f, 1f),
        animationSpec = tween(durationMillis = 1000)
    )
    val ringColor = when {
        consumed < target * 0.8 -> Color(0xFF4CAF50) // 绿
        consumed < target -> Color(0xFFFF9800) // 橙
        else -> Color(0xFFFF44336) // 红
    }
}

```

```

    }
    Canvas(modifier = Modifier.size(200.dp)) {
        drawArc(
            color = Color.LightGray.copy(alpha = 0.3f),
            startAngle = 180f, sweepAngle = 180f,
            useCenter = false, style = Stroke(width = 20.dp.toPx())
        )
        drawArc(
            color = ringColor,
            startAngle = 180f, sweepAngle = progress * 180f,
            useCenter = false, style = Stroke(width = 20.dp.toPx())
        )
    }
}

```

该组件选择半圆弧而非完整圆环，是因为半圆弧有明确的起点和终点，用户一眼即可判断进度比例而非仅凭角度感知。颜色三段式渐变（绿→橙→红）对应不同的摄入状态，使信息传达无需文本辅助。所有数值通过 `animateFloatAsState` 实现平滑动画过渡，避免数值跳变带来的生硬感。

### (3). 后端 Gemini 调用服务：Prompt 工程与容错

Gemini 服务是后端 AI 能力的核心，其设计重点在于如何约束大模型输出结构化、可解析的数据。

```

async def analyze_food_image(image_bytes: bytes, mime_type: str) -> dict:
    if not settings.GEMINI_API_KEY:
        raise ValueError("GEMINI_API_KEY is not configured")

    image_b64 = base64.b64encode(image_bytes).decode()
    payload = {
        "contents": [{
            "parts": [
                {"text": USER_PROMPT},
                {"inlineData": {"mimeType": mime_type, "data": image_b64}}
            ]
        }],
        "systemInstruction": {"parts": [{"text": IMAGE_SYSTEM_PROMPT}]},
        "generationConfig": {
            "responseMimeType": "application/json",
            "temperature": 0.2,
        },
    }

    async with httpx.AsyncClient(timeout=60.0) as client:

```

```

resp = await client.post(
    f"{url}?key={settings.GEMINI_API_KEY}", json=payload
)
resp.raise_for_status()
data = resp.json()

text = data["candidates"][0]["content"]["parts"][0]["text"]
cleaned = text.strip()
if cleaned.startswith("```"):
    cleaned = cleaned.removeprefix("```json").removeprefix("```")
    cleaned = cleaned.removesuffix("```").strip()
return json.loads(cleaned)

```

这段代码有几个设计要点。`responseMimeType: application/json` 将 Gemini 的输出强制约束为合法 JSON，是从根源上避免解析异常的关键配置。`temperature: 0.2` 的低温度设置保证了输出稳定性和可复现性——对于食物分析场景，模型的“创意”没有意义，准确性才是核心。System Prompt 中嵌入精确的 JSON Schema，定义每个字段的类型和取值范围，使 AI 知晓后端预期的数据结构。响应后的清理步骤消除 Gemini 可能添加的 markdown 代码围栏，增加容错能力。整个函数使用 `async httpx` 异步调用并设置 60 秒超时，避免长时间等待阻塞工作线程。

#### (4). 后端营养计算服务：Mifflin-St Jeor 公式

```

def calculate_targets(profile: UserProfile) -> dict:
    bmr = round(10 * profile.weight_kg + 6.25 * profile.height_cm
                - 5 * profile.age + GENDER_COEFFICIENTS[profile.gender])
    tdee = round(bmr * ACTIVITY_FACTORS[profile.activity_level])
    target_calories = round(tdee * HEALTH_GOAL_FACTORS[profile.health_goal])
    protein = round(PROTEIN_FACTOR[profile.health_goal] * profile.weight_kg)
    fat = round(target_calories * 0.25 / 9)
    carbs = max(0, round((target_calories - protein * 4 - fat * 9) / 4))
    return {"bmr": bmr, "tdee": tdee, "target_calories": target_calories,
            "protein": protein, "carbs": carbs, "fat": fat}

```

计算链路分为四步：BMR → TDEE → 目标热量 → 宏量营养素分配。BMR 采用 Mifflin-St Jeor 公式（ $10 \times \text{体重} + 6.25 \times \text{身高} - 5 \times \text{年龄} + \text{性别系数}$ ），这是目前医学界公认最准确的静息代谢估算公式。活动系数和健康目标系数通过字典映射管理，便于扩展。宏量营养素分配中，蛋白质按体重比例计算且不同健康目标使用不同系数（减脂期 1.8g/kg 防肌肉流失，增肌期 2.0g/kg），脂肪固定占 25%，碳水为剩余热量。碳水计算后的  $\max(0, \dots)$  确保极端饮食下不会出现负值。



## (5). 后端 SQL 聚合查询：周统计与日期补全

周统计接口展示了后端处理时间序列数据的典型模式——SQL 聚合加上服务层补全。

```
def get_week_analytics(db, user_id, start, end, metric):
    rows = db.query(
        Meal.meal_date,
        func.coalesce(func.sum(Meal.calories), 0).label("calories"),
        func.coalesce(func.sum(Meal.protein), 0).label("protein"),
        func.coalesce(func.sum(Meal.carbs), 0).label("carbs"),
        func.coalesce(func.sum(Meal.fat), 0).label("fat"),
    ).filter(
        Meal.user_id == user_id, Meal.meal_date.between(start, end)
    ).group_by(Meal.meal_date).order_by(Meal.meal_date).all()

    day_map = {r.meal_date.isoformat(): r for r in rows}
    days = []
    current = start
    while current <= end:
        entry = day_map.get(current.isoformat())
        days.append(DayItem(
            date=current.isoformat(),
            calories=entry.calories if entry else 0,
            protein=entry.protein if entry else 0,
            carbs=entry.carbs if entry else 0,
            fat=entry.fat if entry else 0,
        ))
        current += timedelta(days=1)
    return WeekAnalytics(days=days, ...)
```

关键设计在于直接查 `meals` 表的合计字段而非 JOIN `meal_items` 明细表。这是预计算策略的直接体现——每个餐食在创建或更新时已将明细求和写入主表，统计查询只需扫描 7 天内的少量记录即可完成聚合，无需关联多表。日期补全循环保证无论数据库是否有记录，前端始终收到 7 个数据点，避免因数据缺失导致图表显示异常。`func.coalesce` 确保没有记录的天数返回 0 而非 NULL。

## 3.2 运行截图

### (1). 首页



(2). 食物详情页



(3). 日记页

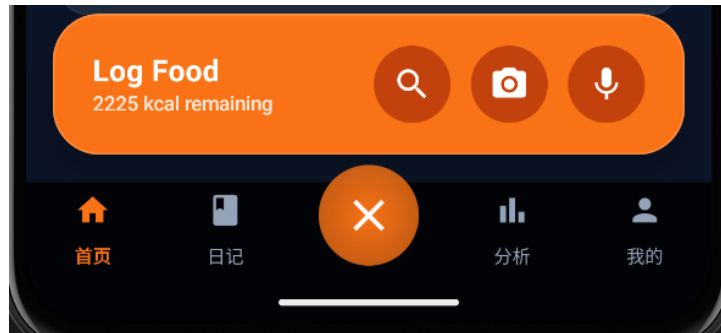


(4). 分析页

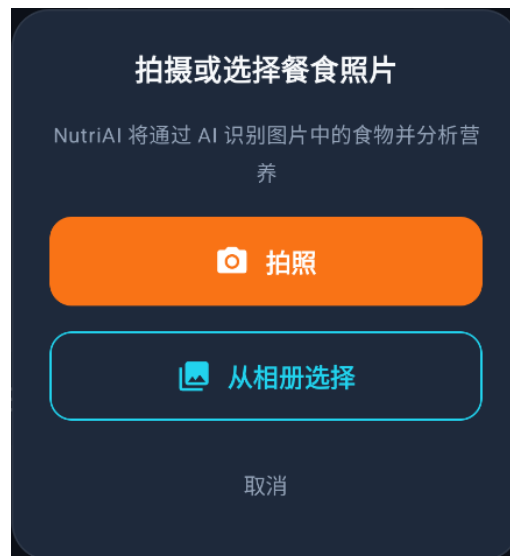


(5). 拍照识别分析餐食

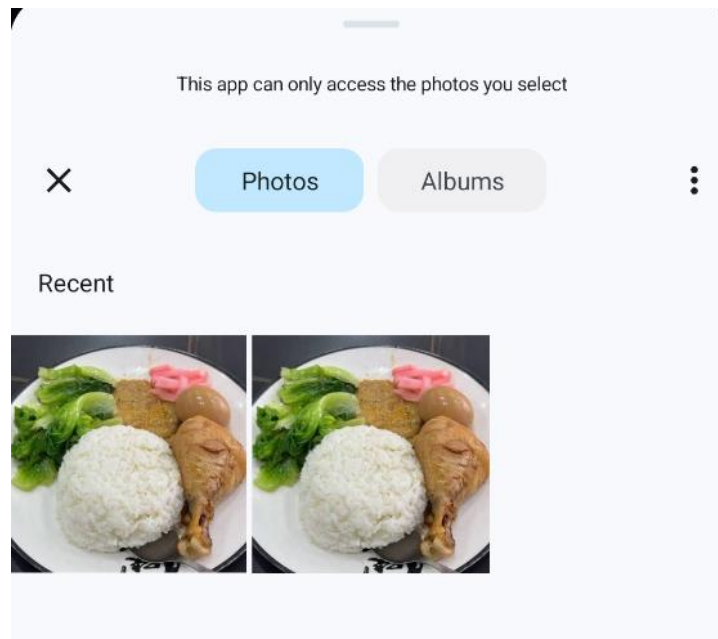
A. 点击加号



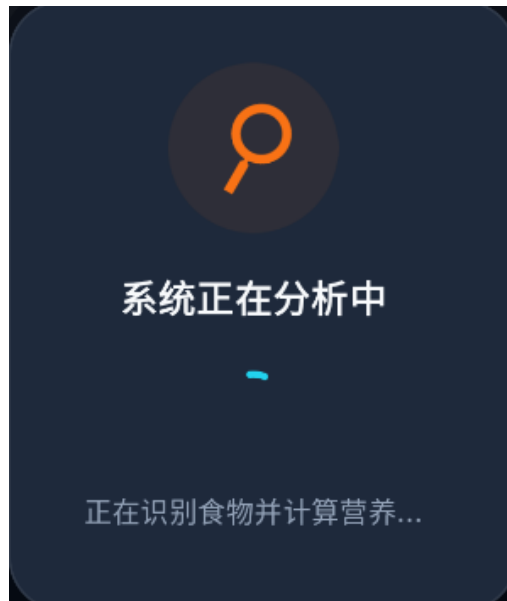
B. 点击从相册选择



C. 选取一张食物图片



D. 正在分析加载



E. 生成餐食详情页



F. 保存后返回主页



## 4. 总结及体会

NutriAI 项目从零到一实现了一个功能完整的 AI 辅助饮食管理应用，覆盖了拍照识别、营养分析、数据记录、趋势统计、图片管理等核心功能链路。在技术层面，项目采用前后端分离架构，前端基于 Jetpack Compose 声明式 UI 框架构建了 Material 3 风格的多页面应用，展示了 Compose 在状态管理、自定义绘制、动画系统和相机集成等方面的成熟度；后端基于 FastAPI 实现了分层清晰、职责明确的 REST API 服务，通过 SQLAlchemy ORM 管理 7 张数据库表的关联关系，利用预计算策略平衡了写入和查询的性能需求。在 AI 集成层面，通过 Gemini 的 JSON Mode 和精确的 System Prompt 工程实现了从非结构化图片到结构化营养数据的端到端自动化转换，其中 System Prompt 的字段定义直接影响 AI 输出质量这一经验，加深了对大模型应用开发的理解。

项目开发过程中的几个技术决策值得总结。UUID 主键策略在前后端分离架构下显著优于自增 ID，前端可在创建时预生成 ID 无需等待数据库返回。预计算策略（写入时合计、查询时直读）虽然增加了写入逻辑的复杂度，但大幅简化了统计查询的实现。图片存储使用本地文件系统加数据库元信息索引的方案在 MVP 阶段足够高效，storage\_key 的 key-value 模式设计为后续迁移对象存储预

留了空间。Android 模拟器与宿主机通信的 `10.0.2.2` 规则是一个典型的平台特定知识，这类问题在跨平台开发中频繁出现，需要建立对应的排查思维。

从工程角度看，项目仍存在若干可优化方向。用户认证系统是当前最显著的功能缺口，MVP 阶段使用的固定测试用户限制了多用户场景的验证。AI 文本分析可以扩展输入方式，支持用户通过文字描述或语音输入识别餐食。AI 周报功能可以进一步发挥大模型的文本生成能力，每周自动生成个性化的饮食总结和建议。图片存储应迁移至对象存储以确保生产环境下的可靠性和访问性能。CI/CD 流水线的建立可以自动化测试和部署流程，减少人工操作引入的错误。

通过 NutriAI 项目的完整开发实践，深入理解了全栈应用的架构设计方法、前后端分离模式下的接口设计规范、AI 大模型的工程化集成思路，以及从用户需求分析到系统设计、编码实现和测试验证的完整项目流程。这些经验对于后续构建更复杂的 AI 驱动应用具有直接的参考价值。